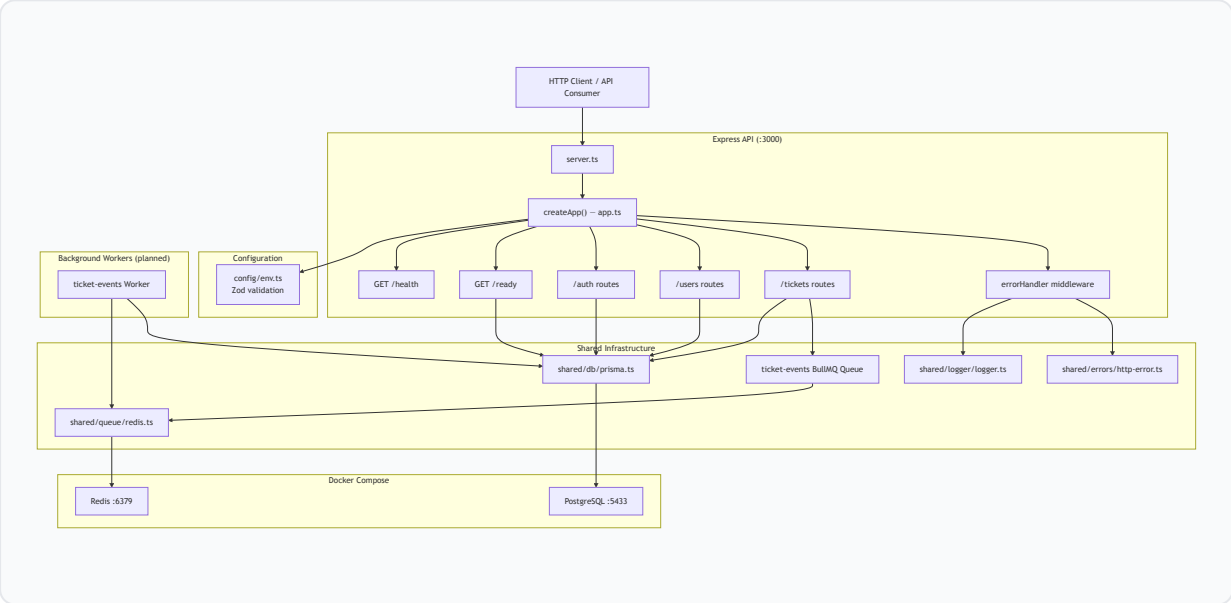
# TicketOps

Target Architecture & Flow Diagrams — Generated July 15, 2026

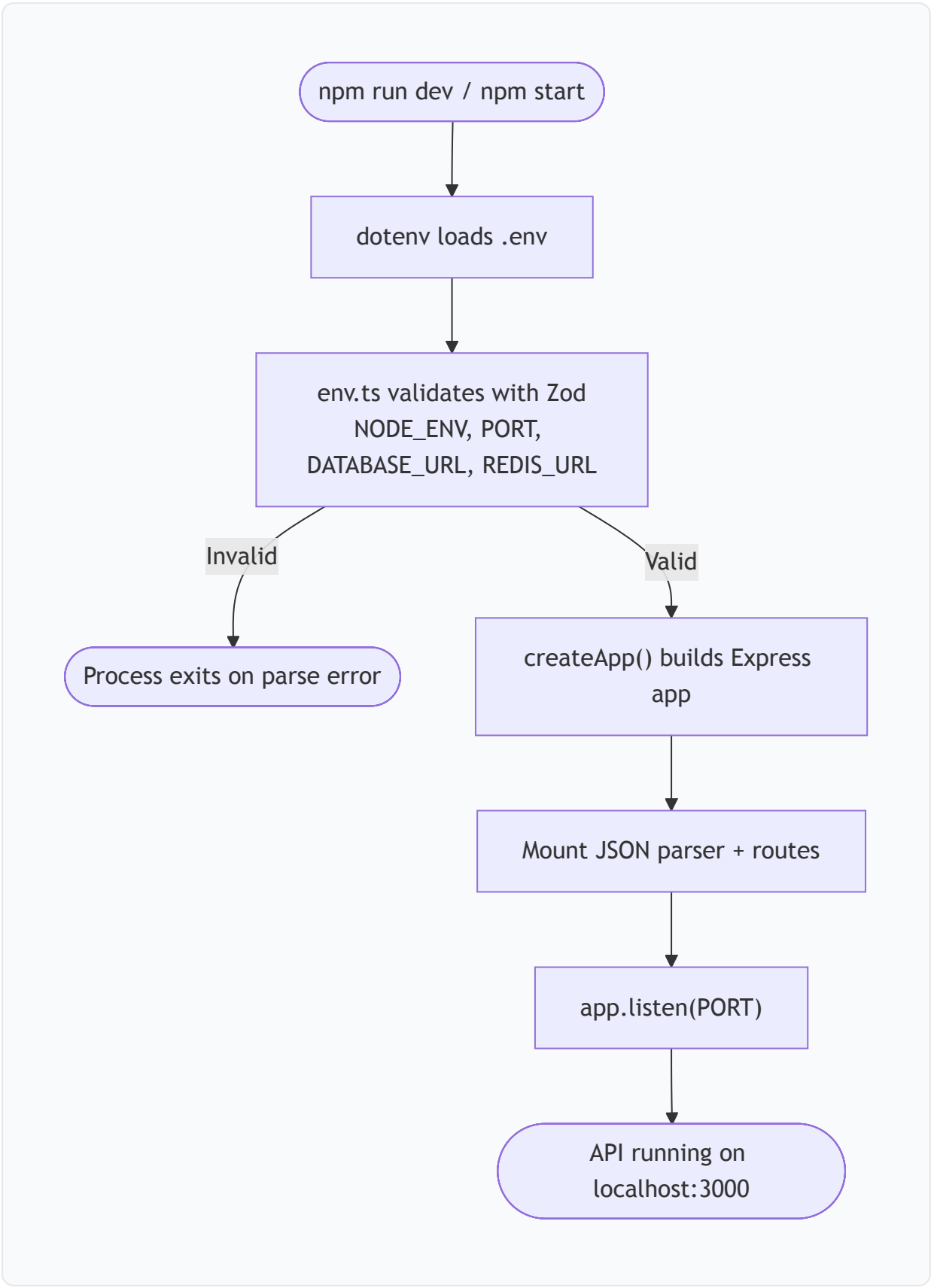
TicketOps is a modular Node.js backend for ticket operations workflows. It uses TypeScript, Express, PostgreSQL, Prisma, Redis, and BullMQ. The diagrams below show the intended system design, with current implementation status noted where relevant.

■ Implemented ■ Planned / stubbed

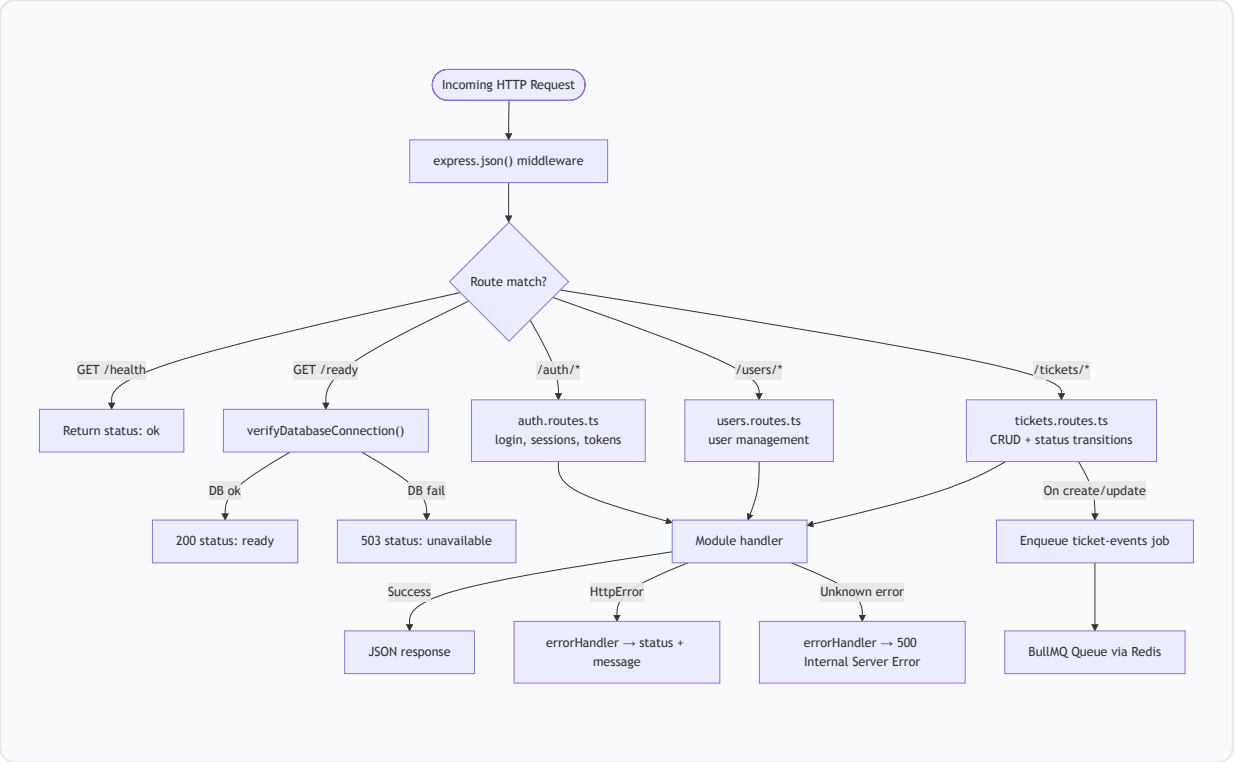
## 1. System Architecture (Target)



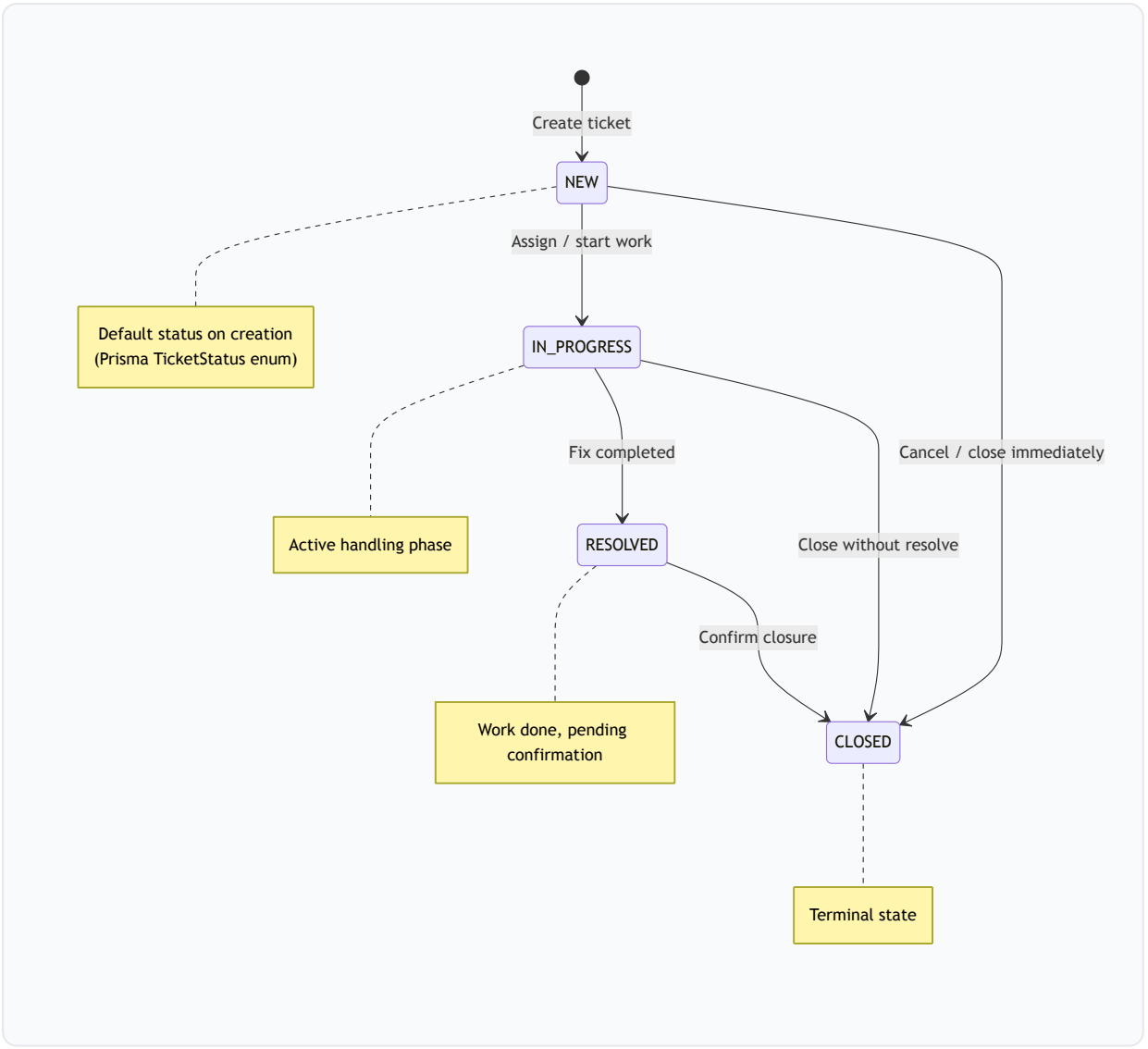
## 2. Application Bootstrap Flow



## 3. Request Handling Flow (Target)

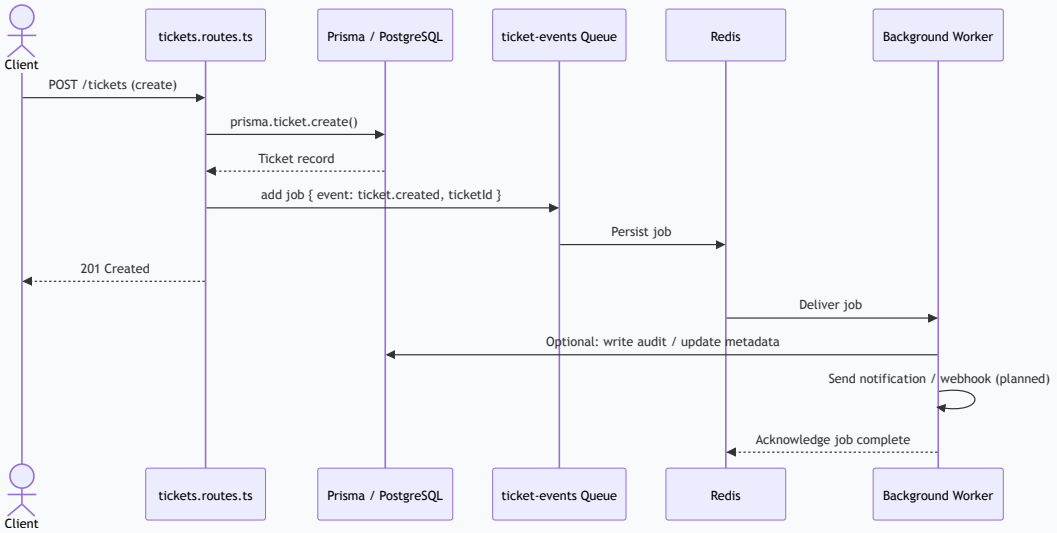


## 4. Ticket Lifecycle Flow

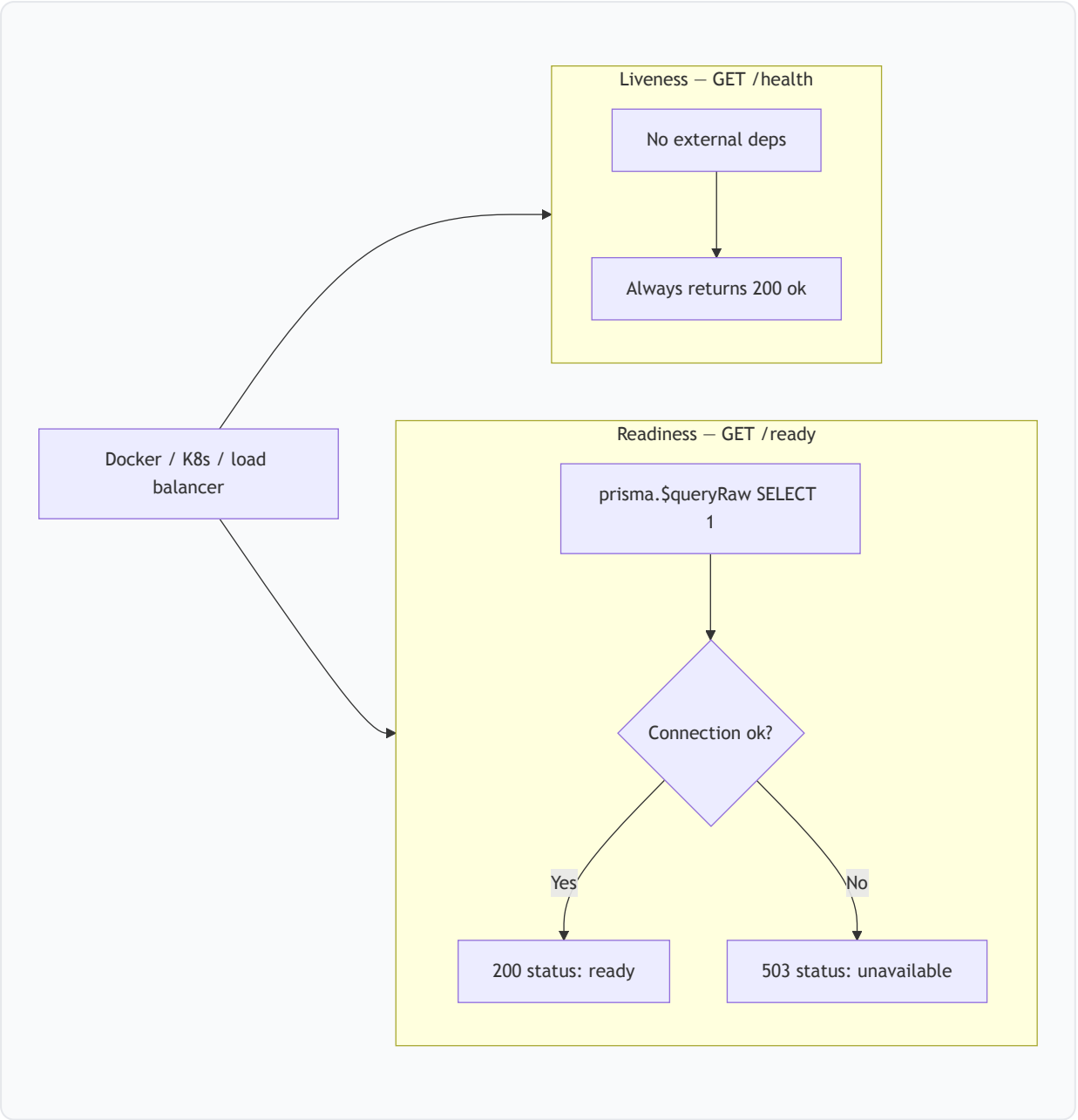



## 5. Ticket Event Pipeline (Target)

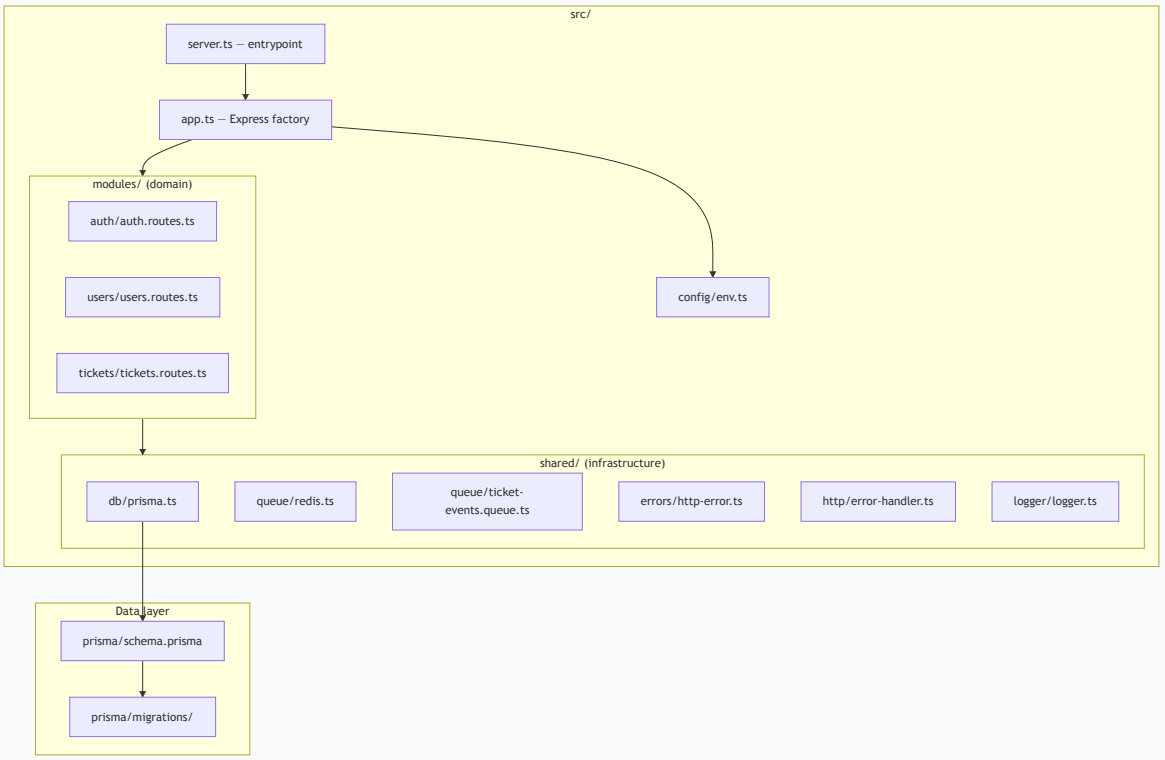
Async side effects (notifications, audit logs, integrations) are decoupled via BullMQ.


Client
tickets.routes.ts
Prisma / PostgreSQL
ticket-events Queue
Redis
Background Worker
POST /tickets (create)
prisma.ticket.create()
Ticket record
add job { event: ticket.created, ticketId }
Persist job
201 Created
Deliver job
Optional: write audit / update metadata
Send notification / webhook (planned)
Acknowledge job complete
Client
tickets.routes.ts
Prisma / PostgreSQL
ticket-events Queue
Redis
Background Worker

## 6. Readiness & Health Checks



## 7. Module Structure




## 8. Data Model

| Ticket | | |
|---|---|---|
| uuid | id | PK |
| string | subject | |
| string | description | |
| TicketStatus | status | |
| datetime | createdAt | |
| datetime | updatedAt | |

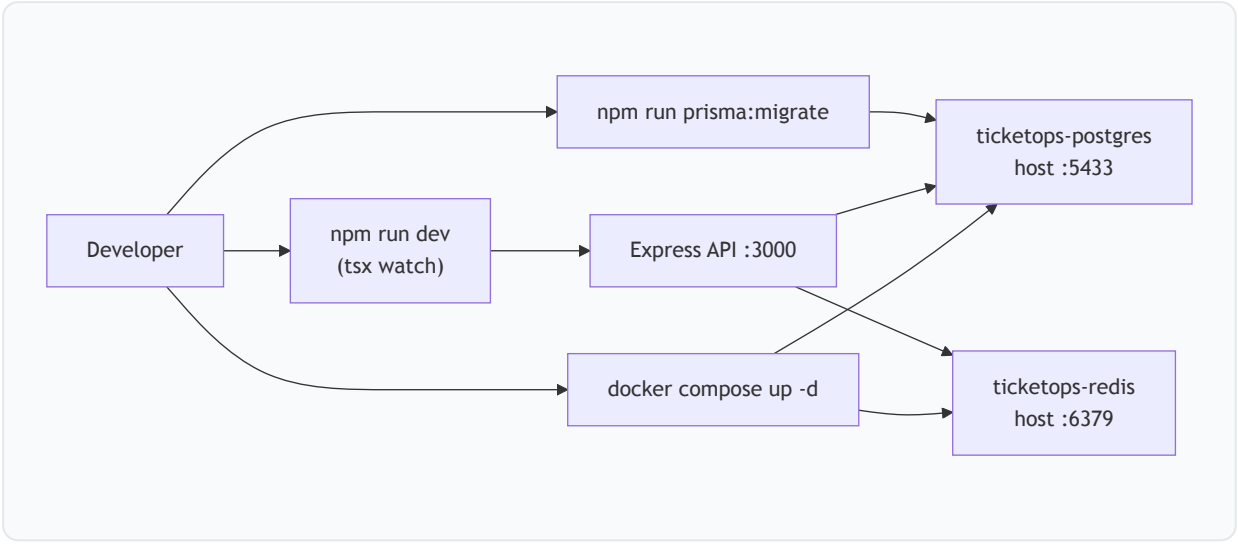
### TicketStatus enum

| Value | Meaning |
|---|---|
| `NEW` | Ticket created, not yet assigned or started |
| `IN_PROGRESS` | Actively being worked on |
| `RESOLVED` | Work completed, awaiting closure |
| `CLOSED` | Terminal state |

## 9. API Endpoints

| Endpoint | Status | Description |
|---|---|---|
| `GET /health` | Implemented | Liveness check — returns `{ status: "ok" }` |
| `GET /ready` | Implemented | Readiness check — verifies PostgreSQL via Prisma |
| `GET /tickets` | Stub | List tickets — currently returns empty array |
| `POST /tickets` | Planned | Create ticket + enqueue ticket-events job |
| `GET /tickets/:id` | Planned | Fetch single ticket |
| `PATCH /tickets/:id` | Planned | Update ticket / change status |
| `/auth/*` | Stub | Authentication routes (router exists, no handlers) |
| `/users/*` | Stub | User management routes (router exists, no handlers) |

## 10. Local Development Stack




## 11. Environment Variables

| Variable | Validated by | Example |
|---|---|---|
| `NODE_ENV` | Zod enum | `development` |
| `PORT` | Zod positive int | `3000` |
| `DATABASE_URL` | Zod URL | `postgresql://ticketops:ticketops@localhost:5433/ticketops` |
| `REDIS_URL` | Zod URL | `redis://localhost:6379` |

## 12. Implementation Status Summary

| Component | Status |
|---|---|
| Express bootstrap ( `server.ts` , `app.ts` ) | Implemented |
| Environment validation ( `config/env.ts` ) | Implemented |
| `GET /health` | Implemented |
| `GET /ready` with Prisma check | Implemented |
| Prisma schema + Ticket model + migrations | Implemented |
| Docker Compose (PostgreSQL + Redis) | Implemented |

| Component | Status |
| --- | --- |
| Redis client + BullMQ queue definition | Defined, not wired |
| Module routers (auth, users, tickets) | Stubbed, not mounted |
| Error handler middleware | Defined, not mounted |
| Background worker for ticket-events | Planned |
| Full ticket CRUD + status transitions | Planned |
| Authentication | Planned |